FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Raven Gale Enriquez
Section:	AN31
Professor:	
Date:	Oct 2, 2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects

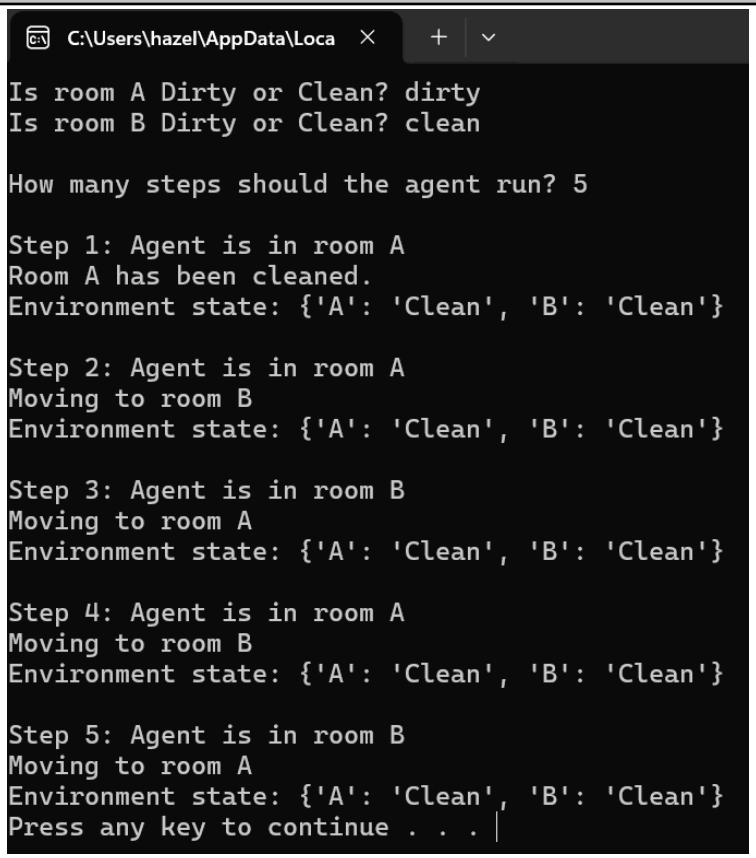
- If-Else statements
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

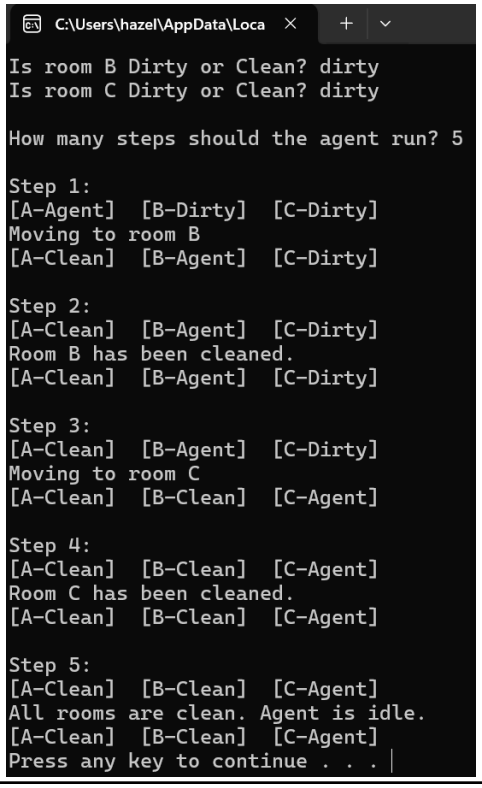
Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

```
# -------- Environment setup --------
rooms = {
   "A": input("Enter initial state of Room A (dirty/clean): ").strip().lower(),
   "B": input("Enter initial state of Room B (dirty/clean): ").strip().lower()
}
```

```
for room in rooms:
    if rooms[room] not in ("dirty", "clean"):
        raise ValueError("Room state must be 'dirty' or 'clean'.")

def is_dirty(room):
    """Return True when the selected room is dirty."""
    return rooms[room] == "dirty"

def clean_room(room):
    """Clean the selected room."""
    rooms[room] = "clean"

class VacuumAgent:
    def __init__(self, start_room="A"):
        self.location = start_room

    def move(self):
        """Switch between Room A and Room B."""
        self.location = "B" if self.location == "A" else "A"
        return f"Moved to Room {self.location}"

    def perceive_and_act(self):
        """
        Apply the two rules:
        1. Dirty -> clean.
        2. Clean -> move.
        """
        if is_dirty(self.location):
            clean_room(self.location)
            action = f"Cleaned Room {self.location}"
        else:
            action = self.move()
        return action

steps = int(input("Enter number of simulation steps: "))
if steps < 0:
    raise ValueError("Number of steps cannot be negative.")

agent = VacuumAgent("A")

print("\nInitial Environment:")
print(f"Room A: {rooms['A']}")
```

```
print(f"Room B: {rooms['B']}")
print(f"Agent location: Room {agent.location}")

print("\nSimulation:")
for step in range(1, steps + 1):
    action = agent.perceive_and_act()
    print(f"Step {step}:")
    print(f"  Location: Room {agent.location}")
    print(f"  Action: {action}")
    print(f"  Environment: {rooms}")

print("\nFinal Environment:")
print(f"Room A: {rooms['A']}")
print(f"Room B: {rooms['B']}")
print(f"Agent location: Room {agent.location}")
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. **What rules does the agent follow?**
   The agent follows simple **if–then rules**:
   - If the current room is dirty, the agent **cleans the room**.
   - If the current room is clean, the agent **moves to the other room**.
   - It continues this process until the task is completed.
2. **How does the agent decide when to clean and when to move?**
   The agent checks the condition of the current room. If it is **dirty**, it performs the cleaning action. If it is **already clean**, it moves to the other room to check it.
3. **What happens if both rooms are already clean at the start?**
   Since there is nothing to clean, the agent will check the rooms and move between them without performing a cleaning action. The environment is already clean.

4. **How would you modify the agent to handle more than two rooms?**
   I would replace the two-room condition with a **list of rooms**. The agent could keep track of its current room and move to the next room in the list. For example, it could check each room one by one, clean it if it is dirty, and continue until all rooms have been checked and cleaned.